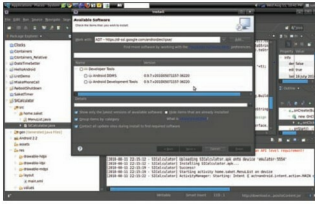

Figure 1: Installing the ADT Plug-in

Android project folder structure

When the new Android project is created on disk, it has a folder structure as shown in Figure 4. This helps you organise your application resources in a logical manner:

Folder/File	Purpose
src	Holds all the Java classes that you create.
res	Short for *Resource*, it holds all your images, layout files, and stringclass="Apple-converted-space" value XML.
drawable	Contains three folders for storing images with different sizes based on the resolution of mobile phone screens.
layout	Consists of a single XML file initially defining the layout configurations.
values	Holds an XML file named *strings.xml* which maps values of IDs to string values. Discussed later in the tutorial.
AndroidManifest.xml	This is the main configuration file for the whole of Android application. It defines the way your application will be launched, the number of activities present in your application, the permissions that your application has, etc.
default.properties	This file is generally used for localisation purposes.

User interface

Let's begin with *res/layout/main.xml*, which configures the layout of the application. The layout terms/tags that we use are very similar to Java Swing components:

- *LinearLayout:* A layout that arranges its children in a single row or column. The orientation attribute specifies whether the layout is horizontal or vertical (a row or a column).
- *Button:* Specifies a push-button.
- *EditText:* Corresponds to an editable text box where the user can enter data.
- *TextView:* Corresponds to a *Label* field.

You can either use the UI layout tool provided by ADT, or hand-edit the XML. I prefer the latter because I have more control over the UI, and the ADT tool is a bit jerky as of now, needing a lot of improvement.

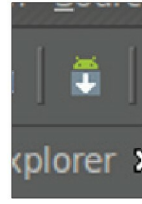

Figure 2: Android SDK and AVD Manager toolbar button

Using the tags listed above, I created the layout of the demo application in *main.xml.* You can obtain the full code from my GitHub repository at *http://goo.gl/4Ni2.* Now, let's examine how we build the layout. As the first step, I used *LinearLayout* to set the layout on the screen:

```
<LinearLayout android:id="@+id/LinearLayout01"
        android:layout_width="fill_parent"
        android:layout_height="fill_parent"
        android:orientation="vertical"
        android:background="@drawable/simpleinterest1"
        xmlns:android="http://schemas.android.com/apk/res/android">
```

Clause	Meaning
android:id	A unique ID for the component, which can be used to refer to it in application code.
android:layout_width android:layout_height	Gives the instruction on how the layout should occupy the screen. The *fill_parent* value fills the screen completely, while *wrap_content* varies size as per the size of contained content.
android:orientation	Orientation of the layout. When *horizontal,* components are added side by side; when *vertical*, components are added one below the other.
android:background	Indicates a background image—a resource in the drawable folder. In this case, it is the file res/drawable-hdpi/simpleinterest1.jpg, which you can obtain from the GitHub repository.

The next step is to add a *TextView* specifying the title, the result of which is shown in Figure 5.

We then add another *LinearLayout* nested inside the parent *LinearLayout*, with horizontal orientation, so that we can have a label and a text-box beside each other, as shown in Figure 6:

```
<LinearLayout android:id="@+id/LinearLayout02"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:layout_marginTop="20dp"
```